

wildfires

May 22, 2026

This is the notebook for my Wildfires project for SDS210.

Our first step is importing all dependencies.

```
[1]: import requests
import pandas as pd
import geopandas as gpd
import time
import datetime
import folium
from folium.plugins import MarkerCluster
from folium.plugins import HeatMap
import numpy as np
import branca.colormap as cm
```

Next step is accessing the FIRMS API and ensuring that we are successful.

The MAP_KEY would be removed and future users would need to add theirs in for it to work.

```
[2]: # We need to access the API and to do that, will use the map key that permits
    ↪ access.
MAP_KEY = '54684dde74a099b139ddb bef0f621891'

# Now let's check how many results we have
url = 'https://firms.modaps.eosdis.nasa.gov/mapserver/mapkey_status/?MAP_KEY='
    ↪ + MAP_KEY
try:
    test_response = requests.get(url)
    test_data = test_response.json()
    test_df = pd.Series(test_data)
    display(test_df)
except:
    # possible error, wrong MAP_KEY value, check for extra quotes, missing letters
    print ("There is an issue with the query. \nTry in your browser: %s" % url)
```

```
transaction_limit      5000
current_transactions    29
transaction_interval    10 minutes
dtype: object
```

```
[3]: # this url will return information about all supported sensors and their
      ↪corresponding datasets
      # instead of 'all' you can specify individual sensor, ex:LANDSAT_NRT
      sensor_data = 'https://firms.modaps.eosdis.nasa.gov/api/data_availability/csv/'
      ↪+ MAP_KEY + '/all'
      test_df = pd.read_csv(sensor_data)
      display(test_df)
```

	data_id	min_date	max_date
0	MODIS_NRT	2026-03-01	2026-05-22
1	MODIS_SP	2000-11-01	2026-02-28
2	VIIRS_NOAA20_NRT	2026-04-01	2026-05-22
3	VIIRS_NOAA20_SP	2018-04-01	2026-03-31
4	VIIRS_NOAA21_NRT	2024-01-17	2026-05-22
5	VIIRS_SNPP_NRT	2026-04-01	2026-05-22
6	VIIRS_SNPP_SP	2012-01-20	2026-03-31
7	LANDSAT_NRT	2022-06-20	2026-05-21
8	GOES_NRT	2022-08-09	2026-05-22
9	BA_MODIS	2000-11-01	2026-02-01
10	BA_VIIRS	2012-03-01	2026-02-01

```
[4]: # We are interested in the wildfires in Australia and so will select this
      ↪information using a bounding box. Aus = 110 -55, 180 -10
      modis_nrt_url = 'https://firms.modaps.eosdis.nasa.gov/api/area/csv/' + MAP_KEY
      ↪+ '/MODIS_NRT/110,-50,160,-11/3'
      modis_nrt_df = pd.read_csv(modis_nrt_url)

      viirs_noaa20_nrt_url = 'https://firms.modaps.eosdis.nasa.gov/api/area/csv/' +
      ↪MAP_KEY + '/VIIRS_NOAA20_NRT/110,-50,160,-11/3'
      viirs_noaa20_nrt_df = pd.read_csv(viirs_noaa20_nrt_url)

      viirs_noaa21_nrt_url = 'https://firms.modaps.eosdis.nasa.gov/api/area/csv/' +
      ↪MAP_KEY + '/VIIRS_NOAA21_NRT/110,-50,160,-11/3'
      viirs_noaa21_nrt_df = pd.read_csv(viirs_noaa21_nrt_url)

      viirs_snpp_nrt_url = 'https://firms.modaps.eosdis.nasa.gov/api/area/csv/' +
      ↪MAP_KEY + '/VIIRS_SNPP_NRT/110,-50,160,-11/3'
      viirs_snpp_nrt_df = pd.read_csv(viirs_snpp_nrt_url)
```

```
[5]: # Create a funtion that checks time of aquisition and calculates time delta
      def add_time_since_acq(df):
          df["acq_date"] = pd.to_datetime(df["acq_date"])    # Ensuring date is in
          ↪datetime format
          df["acq_datetime"] = pd.to_datetime(
              df["acq_date"].astype(str) + df["acq_time"].astype(str).str.zfill(4),
              ↪# Turns all values into 4 digit HHHH format
              format = "%Y-%m-%d%H%M"
```

```

        ).dt.tz_localize('UTC')

    # Get current time in UTC
    current_time = pd.Timestamp.now('UTC')
    df["time_since_detection"] = current_time - df["acq_datetime"]

    # Calculate hours since detection
    df["hours_since_detection"] = df["time_since_detection"].dt.total_seconds()/
↪3600
    df["hours_since_detection"] = df["hours_since_detection"].round(1)

    # Convert data types to string format so folium can take them
    df["acq_date_str"] = df["acq_date"].astype(str)
    df["time_since_detection_str"] = df["time_since_detection"].astype(str)
    df["acq_datetime_str"] = df["acq_datetime"].astype(str)

    # Drop datetime columns and "time_since_detection_str" as it is covered by ↪
↪"hours_since_detection".
    df = df.drop(columns=["time_since_detection", "time_since_detection_str", ↪
↪"acq_date", "acq_datetime"])

    return df

# Apply to each of our datasets
modis_nrt_df = add_time_since_acq(modis_nrt_df)
viirs_noaa20_nrt_df = add_time_since_acq(viirs_noaa20_nrt_df)
viirs_noaa21_nrt_df = add_time_since_acq(viirs_noaa21_nrt_df)
viirs_snpp_nrt_df = add_time_since_acq(viirs_snpp_nrt_df)

```

[6]: # Convert each dataset to GeoDataFrame and apply EPSG=4326

```

modis_nrt_gdf = gpd.GeoDataFrame(
    modis_nrt_df,
    geometry=gpd.points_from_xy(
        modis_nrt_df["longitude"],
        modis_nrt_df["latitude"]
    ),
    crs="EPSG:4326")

viirs_noaa20_nrt_gdf = gpd.GeoDataFrame(
    viirs_noaa20_nrt_df,
    geometry=gpd.points_from_xy(
        viirs_noaa20_nrt_df["longitude"],
        viirs_noaa20_nrt_df["latitude"]
    ),
    crs="EPSG:4326")

```

```

viirs_noaa21_nrt_gdf = gpd.GeoDataFrame(
    viirs_noaa21_nrt_df,
    geometry=gpd.points_from_xy(
        viirs_noaa21_nrt_df["longitude"],
        viirs_noaa21_nrt_df["latitude"]
    ),
    crs="EPSG:4326")

viirs_snpp_nrt_gdf = gpd.GeoDataFrame(
    viirs_snpp_nrt_df,
    geometry=gpd.points_from_xy(
        viirs_snpp_nrt_df["longitude"],
        viirs_snpp_nrt_df["latitude"]
    ),
    crs="EPSG:4326")

```

Now we want to take our four GeoDataFrames and add them to a Folium map.

```

[7]: # 1. Initialize the Folium map
australia_bushfire_map = folium.Map(location=[-28.281828, 136.145401],
    ↪zoom_start=5)

# 2. Add multiple datasets using GeoPandas

modis_nrt_gdf.explore(
    m=australia_bushfire_map,
    column='hours_since_detection',
    name='MODIS NRT',
    tooltip=['confidence', 'hours_since_detection'],
    cmap='YlOrRd_r',
    style_kwds={'fillOpacity': 0.4, 'color': 'white', 'weight': 0.1},
    show=True
)

viirs_noaa20_nrt_gdf.explore(
    m=australia_bushfire_map,
    column='hours_since_detection',
    name='VIIRS NOAA20 NRT',
    tooltip=['confidence', 'hours_since_detection'],
    cmap='YlOrRd_r',
    style_kwds={'fillOpacity': 0.4, 'color': 'white', 'weight': 0.1},
    legend=False,
    show=True
)

viirs_noaa21_nrt_gdf.explore(
    m=australia_bushfire_map,

```

```

        column='hours_since_detection',
        name='VIIRS NOAA21 NRT',
        tooltip=['confidence', 'hours_since_detection'],
        cmap='YlOrRd_r',
        style_kwds={'fillOpacity': 0.4, 'color': 'white', 'weight': 0.1},
        legend=False,
        show=True
    )

viirs_snpp_nrt_gdf.explore(
    m=australia_bushfire_map,
    column='hours_since_detection',
    name='VIIRS SNPP NRT',
    tooltip=['confidence', 'hours_since_detection'],
    cmap='YlOrRd_r',
    style_kwds={'fillOpacity': 0.4, 'color': 'white', 'weight': 0.1},
    legend=False,
    show=True
)

# 3. Add Folium plugins back on top
folium.LayerControl().add_to(australia_bushfire_map)

# Show folium map in notebook
australia_bushfire_map

```

[7]: <folium.folium.Map at 0x1bab5833470>

```

[8]: # Saving map for viewer access
australia_bushfire_map.save("Outputs/australia_bushfire_map.html")

```

Now we will create a choropleth map to show numbers of fire detections per Local Government Area (LGA). Firstly, we need to add our data and manipulate it into useful information.

```

[9]: # Add LGAs data to the project and convert to EPSG:4326
aus_lgas = gpd.read_file("data/LGA_2025_AUST_GDA2020/LGA_2025_AUST_GDA2020.
↳shp").to_crs(epsg=4326)

```

```

[10]: # We will do a spatial join to count fire detections by LGA

# Create dictionary of the four fire detection sources
satellite_sources = {
    "modis_counts": modis_nrt_gdf,
    "noaa20_counts": viirs_noaa20_nrt_gdf,
    "noaa21_counts": viirs_noaa21_nrt_gdf,
    "snpp_counts": viirs_snpp_nrt_gdf
}

```

```

# For loop to spatially join
for col_name, gdf in satellite_sources.items():
    joined = gpd.sjoin(gdf, aus_lgas[["LGA_NAME25", "geometry"]], how="inner",
        predicate="within")
    join_key = "LGA_NAME25_right" if "LGA_NAME25_right" in joined.columns else
        "LGA_NAME25"
    counts = joined.groupby(join_key).size().reset_index(name=col_name)
    counts.rename(columns={join_key: "LGA_NAME25"}, inplace=True)
    aus_lgas = aus_lgas.merge(counts, on="LGA_NAME25", how="left")

# Fill NAs
aus_lgas[list(satellite_sources.keys())] = aus_lgas[list(satellite_sources.
    keys())].fillna(0)

# Sum fire detection counts
aus_lgas["fire_count"] = aus_lgas[list(satellite_sources.keys())].sum(axis=1)

```

We now can initialise the map and add our data.

```

[11]: # Initialise the map
bushfire_choropleth = folium.Map(
    location=[-28.281828, 136.145401], zoom_start=5, tiles="CartoDB Positron No
        Labels"
)

# Choropleth map was taking a long time to load so reduced the file to only
    relevant columns for map
aus_lgas_reduced = aus_lgas[["LGA_NAME25", "AREASQKM", "fire_count", "geometry"]]

#simplify geometry
aus_lgas_reduced["geometry"] = aus_lgas_reduced["geometry"].
    simplify(tolerance=0.001, preserve_topology=True)

# Removed the attributes with no geometry, was causing an error
aus_lgas_reduced = aus_lgas_reduced[aus_lgas_reduced.geometry.notna()]

aus_lgas_reduced.head()

# 1. Area Choropleth (Hidden by default using show=False)
folium.Choropleth(
    geo_data=aus_lgas_reduced,
    name="LGA Area (km2)",
    data=aus_lgas_reduced,
    columns=["LGA_NAME25", "AREASQKM"],
    #bins=[0, 500, 2500, 7000, 32000, 62000, 620000],

```

```

    key_on="feature.properties.LGA_NAME25",
    fill_color="Blues",
    fill_opacity=0.8,
    line_opacity=0.2,
    legend_name="Area (km²)    0 → 500 → 2.5k → 7k → 32k → 620k",
    show=False, # Keeps the map clean on initial load
).add_to(bushfire_choropleth)

# 2. Detection count Choropleth (Visible by default)
folium.Choropleth(
    geo_data=aus_lgas_reduced,
    name="Fire Detections per LGA",
    data=aus_lgas_reduced,
    columns=["LGA_NAME25", "fire_count"],
    # As the number of detections can change, the upper bin must be large
    ↪ enough to always encapsulate the data
    #bins=[0, 10, 50, 200, 500, 1000, 10000],
    key_on="feature.properties.LGA_NAME25",
    fill_color="YlOrRd",
    fill_opacity=0.6,
    line_opacity=0.2,
    legend_name="Fire Detections per LGA",
).add_to(bushfire_choropleth)

# 3. Add Interactive Tooltips via an Invisible GeoJson Layer
folium.GeoJson(
    aus_lgas_reduced,
    name="Interactive Tooltips",
    # Make the polygons completely transparent so they do not hide the
    ↪ choropleth colors
    style_function=lambda x: {"fillColor": "#ffffff00", "color": "#ffffff00"},
    tooltip=folium.GeoJsonTooltip(
        fields=["LGA_NAME25", "AREASQKM", "fire_count"],
        aliases=["LGA", "Area (km2)", "Fire Count:"],
        localize=True,
    ),
).add_to(bushfire_choropleth)

# Add Layer Control and display
folium.LayerControl().add_to(bushfire_choropleth)

bushfire_choropleth

```

[11]: <folium.folium.Map at 0x1babb2cdeb0>

```
[12]: # Saving map for viewer access
bushfire_choropleth.save("Outputs/bushfire_choropleth.html")
```

Our final output will show bushfire detections and the closest fire station from the last three hours.

```
[13]: # Finally, we will create a text output that refreshes and shows bushfire
      ↪ detections from the last three hours.
      # It will also assign the closest Rural Fire Station to each detection.

      # Add Rural Fire Station data to project. Reproject to metre-based CRS for
      ↪ distance calculations
aus_rfs = gpd.read_file("data/Rural_Country_Fire_Service_Facilities.geojson").
      ↪ to_crs(epsg=3112)

      # Add a source column and then combine our individual geodataframes
modis_nrt_gdf["source"] = "MODIS_NRT"
viirs_noaa20_nrt_gdf["source"] = "VIIRS_NOAA20_NRT"
viirs_noaa21_nrt_gdf["source"] = "VIIRS_NOAA21_NRT"
viirs_snpp_nrt_gdf["source"] = "VIIRS_SNPP_NRT"

all_fires = gpd.GeoDataFrame(pd.concat([modis_nrt_gdf, viirs_noaa20_nrt_gdf,
      ↪ viirs_noaa21_nrt_gdf, viirs_snpp_nrt_gdf]))
all_fires = all_fires.to_crs(epsg=3112) # Again project to meter based CRS

      # Join to the nearest fire station
first_rfs_response = aus_rfs.sjoin_nearest(
    all_fires, how="left", distance_col="distance_m"
)

      # Convert to KM
first_rfs_response["distance_km"] = (first_rfs_response["distance_m"] / 1000).
      ↪ round(2)

      # Ensure only detections from within the previous three hours are selected
first_rfs_response_3hrs =
      ↪ first_rfs_response[first_rfs_response["hours_since_detection"] < 3]

first_rfs_response_3hrs = first_rfs_response_3hrs[['facility_name',
      ↪ 'hours_since_detection', 'distance_km']]

      # Convert to .html
first_rfs_response_3hrs_html = first_rfs_response_3hrs.to_html()
      # Add timestamp
refreshed_at = pd.Timestamp.now('UTC').strftime('%Y-%m-%d %H:%M UTC')

      # Add line that says when it was last refreshed
first_rfs_response_3hrs_html = f"""
```



```
<h2>Bushfire Detections in the last 3 hours</h2>
<p><strong>Last refreshed:</strong> {refreshed_at}</p>
{first_rfs_response_3hrs.to_html(index=False)}
"""

# Save output
with open("Outputs/first_rfs_response_3hrs.html", "w") as f:
    f.write(first_rfs_response_3hrs_html)
```